27.移除元素

leetcode 链接

0.1 一、题目描述

给你一个数组 nums 和一个值 val，你需要 原地 移除所有数值等于 val 的元素。元素的顺序可能发生改变。然后返回 nums 中与 val 不同的元素的数量。假设 nums 中不等于val 的元素数量为 k，要通过此题，您需要执行以下操作： 更改 nums 数组，使 nums 的前 k 个元素包含不等于 val 的元素。nums 的其余元素和 nums 的大小并不重要。返回 k。

0.1 二、解题思路

双指针法： 由于题目要求删除数组中等于 val 的元素，因此输出数组的长度一定小于等于输入数组的长度，我们可以把输出的数组直接写在输入数组上。 可以使用双指针：右指针right 指向当前将要处理的元素（代码中使用下标i表示），左指针 left（代码中使用 idx 表示） 指向下一个将要赋值的位置。如果右指针指向的元素不等于 val，它一定是输出数组的一个元素，我们就将右指针指向的元素复制到左指针位置，然后将左右指针同时右移；如果右指针指向的元素等于 val，它不能在输出数组里，此时左指针不动，右指针右移一位。整个过程保持不变的性质是：区间 [0,left) 中的元素都不等于 val。当左右指针遍历完输入数组以后，left 的值就是输出数组的长度。这样的算法在最坏情况下（输入数组中没有元素等于 val），左右指针各遍历了数组一次。
时间复杂度：$O(n)$ 空间复杂度：$O(1)$

0.1 三、代码实现

```
int removeElement(vector<int>& nums, int val) {
        int cur = 0, n = nums.size();
        for (int i = 0; i < n; i++) {
            if (nums[i] != val) {
                nums[cur++] = nums[i];
            }
        }
        return cur;
    }
```

977.有序数组的平方

leetcode 链接

0.1 一、题目描述

给你一个按 非递减顺序 排序的整数数组 nums，返回 每个数字的平方 组成的新数组，要求也按 非递减顺序 排序

> 示例 1：
> 输入：nums = [-4,-1,0,3,10]
> 输出：[0,1,9,16,100]
> 解释：平方后，数组变为 [16,1,0,9,100], 排序后，数组变为 [0,1,9,16,100]

0.1 二、解题思路

双指针法：可以使用两个指针分别指向位置 0 和 n−1，每次比较两个指针对应的数，选择较大的那个逆序放入答案并移动指针。

- 时间复杂度：$O(n)$
- 空间复杂度：$O(1)$

0.1 三、代码实现

```
vector<int> sortedSquares(vector<int>& nums) {
        int n = nums.size();
        int left = 0, right = n - 1, cur = n - 1;
        vector<int> res(n);
        while (left <= right) {
            int leftval=nums[left] * nums[left];
            int rightval=nums[right] * nums[right];
            if (leftval <= rightval) {
                res[cur--] = rightval;
                right--;
            } else {
                res[cur--] = leftval;
                left++;
            }
        }
        return res;
    }
```

283.移动零

leetcode 链接

0.1 一、题目描述

给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。

0.1 二、解题思路

双指针法

0.1 三、代码实现

```
void moveZeroes(vector<int>& nums) {
        int n = nums.size(), cur = 0;
        for (int i = 0; i < n; i++) {
            if (nums[i] != 0) {
                swap(nums[i], nums[cur++]);
            }
        }
    }
```

15.三数之和

leetcode 链接

0.1 一、题目描述

给你一个整数数组 nums，判断是否存在三元组 [nums[i], nums[j], nums[k]] 满足 i != j、i != k 且 j != k，同时还满足 nums[i] + nums[j] + nums[k] == 0 。请你返回所有和为 0 且不重复的三元组。注意：答案中不可以包含重复的三元组。

0.1 二、解题思路

排序 + 双指针
首先对数组进行排序，可以处理重复问题，然后三重遍历数组，每一个二重遍历可以看做是两数之和问题，由于排序可以使用双指针处理，通用可以使用 hashtable处理
时间复杂度：$O(N^2)$ 空间复杂度：$O(logN)$

0.1 三、代码实现

```cpp
vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        int n = nums.size();
        vector<vector<int>> res;
        for (int i = 0; i < n; i++) {
            if (nums[i] > 0) {//如果第一个数是正数，那么不可能存在
                break;
            }
            if (i > 0 && nums[i] == nums[i - 1]) {//过滤重复数
                continue;
            }
            int j = i + 1, k = n - 1;
            while (j < k) {
                int sum = nums[i] + nums[j] + nums[k];
                if (sum == 0) {
                    res.push_back({nums[i], nums[j], nums[k]});
                    while (j < k && nums[j] == nums[j + 1]) j++;//过滤重复数
                    while (j < k && nums[k] == nums[k - 1]) k--;
                    j++, k--;
                } else if (sum > 0) {
                    k--;
                } else {
                    j++;
                }
            }
        }
        return res;
    }
```

18.四数之和

leetcode 链接

0.1 一、题目描述

给你一个由 n 个整数组成的数组 nums，和一个目标值 target。请你找出并返回满足下述全部条件且不重复的四元组 [nums[a], nums[b], nums[c], nums[d]] （若两个四元组元素一一对应，则认为两个四元组重复）：

- 0 <= a, b, c, d < n
- a、b、c 和 d 互不相同
- nums[a] + nums[b] + nums[c] + nums[d] == target
- 你可以按任意顺序返回答案

0.1 二、解题思路

排序 + 双指针
「双指针」，对于递增的数组，当我们需要枚举数组中的两个元素时，如果我们发现随着第一个元素的递增，第二个元素是递减的，那么就可以使用双指针的方法
时间复杂度：$O(N^3)$
空间复杂度：$O(logN)$

0.1 三、代码实现

```
vector<vector<int>> fourSum(vector<int>& nums, int target) {
        int n = nums.size();
        vector<vector<int>> res;
        if (n < 4) {
            return res;
        }
        sort(nums.begin(), nums.end());
        for (int i = 0; i < n - 3; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }
            if ((long)nums[i] + nums[i + 1] + nums[i + 2] + nums[i + 3] > target) {
                break;
            }
            if ((long)nums[i] + nums[n - 3] + nums[n - 2] + nums[n - 1] < target) {
                continue;
            }
            for (int j = i + 1; j < n - 2; j++) {
                if (j > i + 1 && nums[j] == nums[j - 1]) {
                    continue;
                }
                if ((long)nums[i] + nums[j] + nums[j + 1] + nums[j + 2] > target) {
                    break;
                }
                if ((long)nums[i] + nums[j] + nums[n - 2] + nums[n - 1] < target) {
                    continue;
                }
                int left = j + 1, right = n - 1;
                while (left < right) {
                    long sum =(long)nums[i] + nums[j] + nums[left] + nums[right];
                    if (sum == target) {
                        res.push_back({nums[i], nums[j], nums[left], nums[right]});
                        while (left < right && nums[left] == nums[left + 1])
                            left++;
                        while (left < right && nums[right] == nums[right - 1])
                            right--;
                        left++;
                        right--;
                    } else if (sum > target) {
                        right--;
                    } else {
```

```
                        left++;
                    }
                }
            }
        }
        return res;
    }
```

75.颜色分类

leetcode 链接

0.1 一、题目描述

给定一个包含红色、白色和蓝色、共 n 个元素的数组 nums ，原地 对它们进行排序，使得相同颜色的元素相邻，并按照红色、白色、蓝色顺序排列。
我们使用整数 0、 1 和 2 分别表示红色、白色和蓝色。
必须在不使用库内置的 sort 函数的情况下解决这个问题。
示例 1：
输入：nums = [2,0,2,1,1,0]
输出：[0,0,1,1,2,2]

0.1 二、解题思路

方法一：计数排序 时间复杂度：$O(N*K)$ 空间复杂度：$O(K)$
方法二：双指针
使用 p0, p1分别执行 0 和 1 应该插入的位置，初始都是 0，然后遍历数组，遇到 0与p0交换位置，遇到1与p1交换位置；注意，在遇到 0 时交换位置时，把1交换到 i 的位置，需要在与 p1 进行交换位置。
时间复杂度：$O(N)$ 空间复杂度：$O(1)$

0.1 三、代码实现

```
void sortColors(vector<int>& nums) {
        int n = nums.size();
        vector<int> cnt(3, 0);
        for (int num : nums) {
            cnt[num]++;
        }
        int idx = 0;
        for (int i = 0; i < 3; i++) {
            while (cnt[i]) {
                nums[idx++] = i;
                cnt[i]--;
            }
        }
    }
```

```
void sortColors(vector<int>& nums) {
        int n = nums.size();
        int left = 0, right = n - 1, cur = 0;
        while (cur <= right) {
            if (nums[cur] == 0) {
                swap(nums[cur], nums[left]);
                cur++;
                left++;
            } else if (nums[cur] == 2) {
                swap(nums[cur], nums[right]);
```

```
                right--;
            } else {
                cur++;
            }
        }
    }
```

11.盛最多水的容器

leetcode 链接

0.1 一、题目描述

给定一个长度为 n 的整数数组 height 。有 n 条垂线，第 i 条线的两个端点是 (i, 0) 和 (i, height[i]) 。
找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。
返回容器可以储存的最大水量。

0.1 二、解题思路

双指针，使用 left，right 指针指向左右边界，每次移动低高度的指针。每次移动计算面积，并更新最大面积
计算公式 (right-left)*height

0.1 三、代码实现

```

int maxArea(vector<int>& height) {
        int n = height.size();
        int left = 0, right = n - 1;
        int maxa = 0;
        while (left < right) {
            maxa = max(maxa, min(height[left], height[right]) * (right - left));
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return maxa;
    }
```